lab2

資科工碩一 313551027 曹咏萱

Part1.

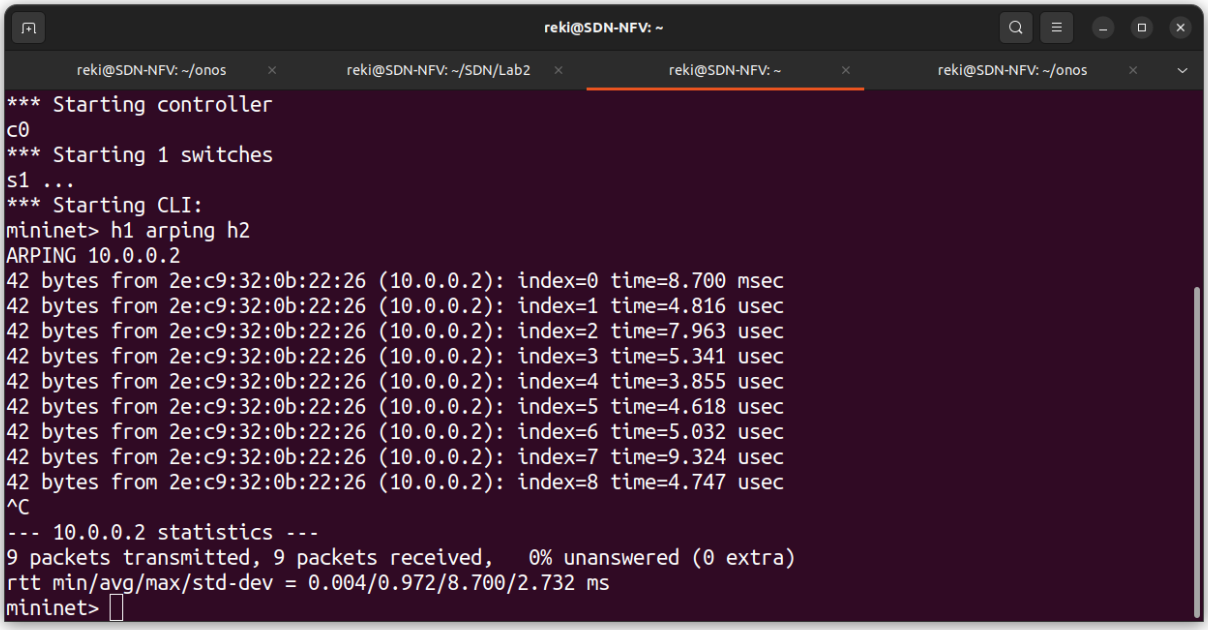
There are 5 distinct “OFPT_FLOW_MOD” headers during the experiment.

Match fields	Actions	Timeout values
OFPXMT_OMP_ETH_TYPE = Unkown	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = ARP	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = 802.1 LLDP	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = IPv4	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_IN_PORT = 1	Output Port = 2	0

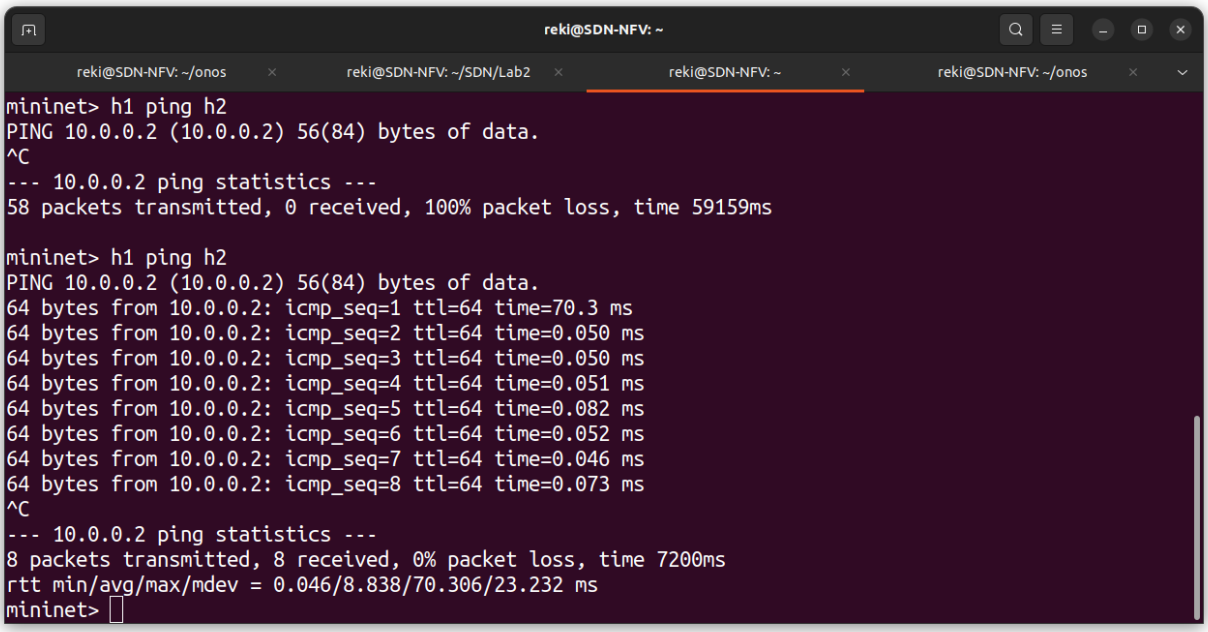
The "Unknown" type may indicate an unrecognized message type or a missing handler for a specific OpenFlow version or message.

Part2

1.

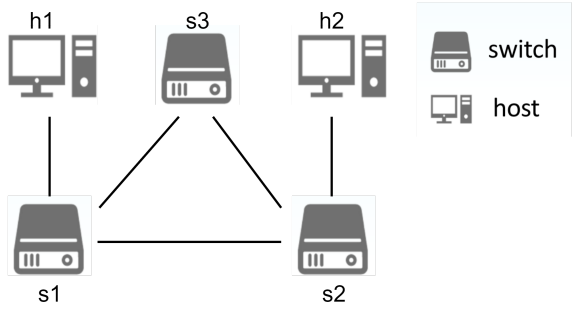


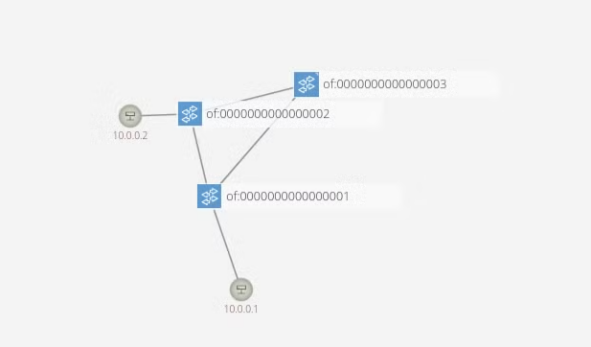
2.



Part3

Create a topology with a loop between switches (h1, h2, h3) to cause broadcast storm.





Install the following flow to each switch and let h1 ping h2.

- Match Fields
  - Ethernet type (ARP)
- Actions
  - Forwarding ARP packets to all port

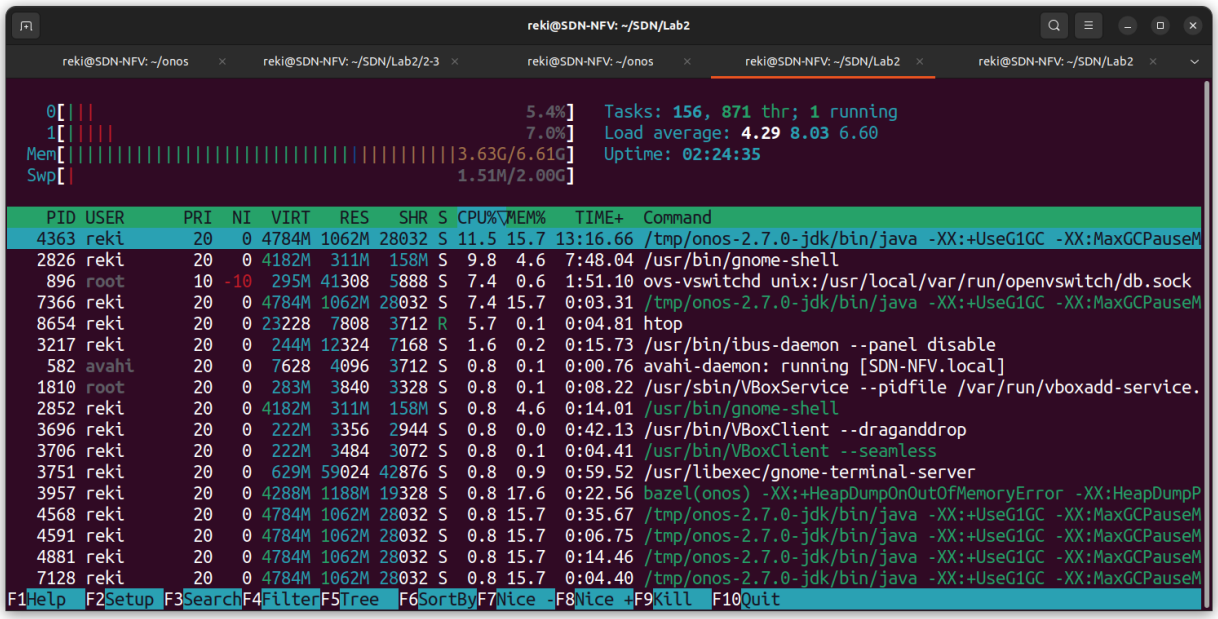
```
mininet> h1 arping h2 # send ARP request
```

Then, h1 will start sending the packet to s1.

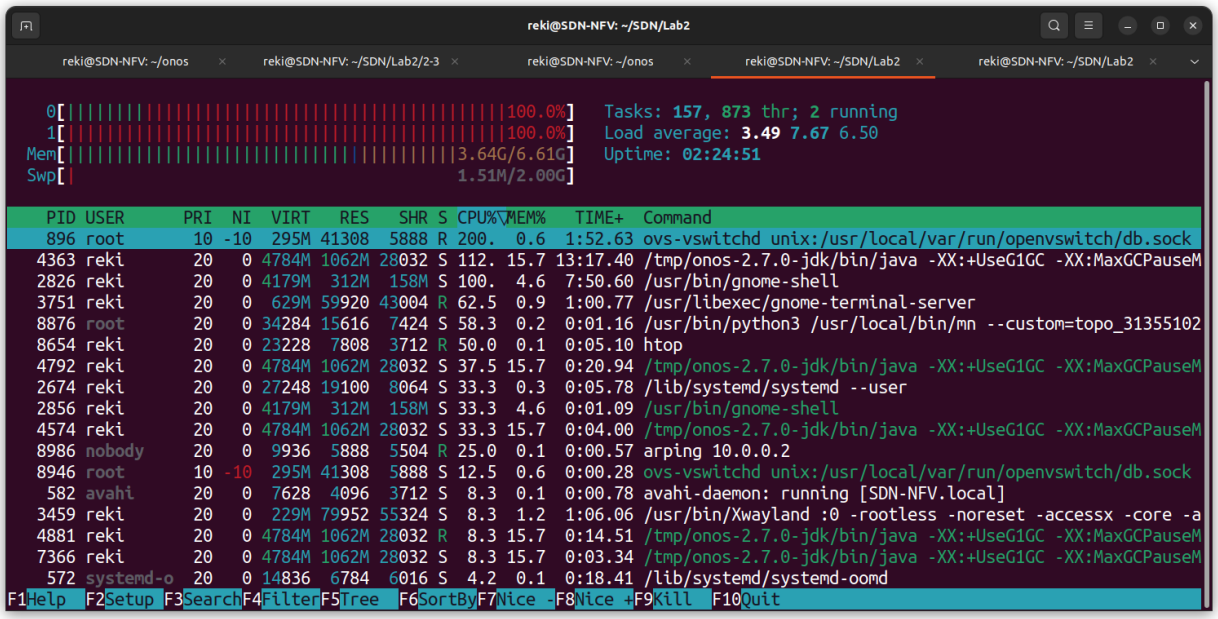
1. Since it is a broadcast packet, s1 copies one and sends it to s2 and s3.
2. When s3 receives the packet it will send it to s3 only, cause it was from s1.
3. At the same time, s2 also receives a packet and starts sending it to s3 and h2.

After the ping, two packets will be retained on the internet. While Layer 2 doesn't have the TTL (Time to Live) mechanism, retained packets will keep increasing and waste a large amount of resources.

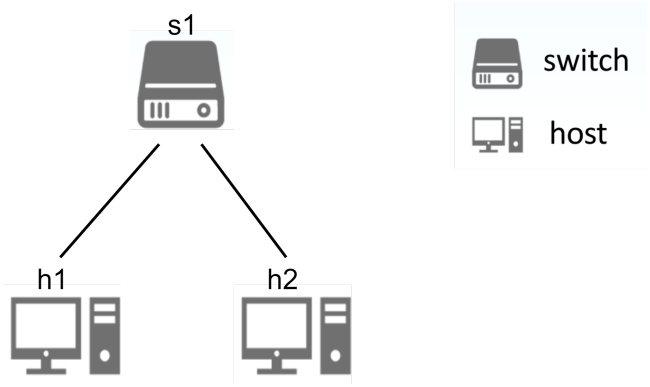
The CPU utilization before sending packets.



The CPU utilization after sending packets.



Part4



1. [Data Plane] h1 sends the packet to s1.
2. [Data Plane] Since s1 doesn't have a flow rule for forwarding this packet, it generates a PACKET-IN message and sends the packet to the ONOS controller.
3. [Control Plane] Reactive Forwarding Application(org.onosproject.fwd) receives PACKET-IN and executes the function process() to decide the path of forwarding the packet.

```
public void process(PacketContext context){
    ...
    MacAddress macAddress = ethPkt.getSourceMAC();
    ReactiveForwardMetrics macMetrics = null;
    macMetrics = createCounter(macAddress);
    inPacket(macMetrics);
    ...

    // get a set of paths that lead from here to the destination edge switch
    Set<Path> paths =
            topologyService.getPaths(topologyService.currentTopology(),
                                     pkt.receivedFrom().deviceId(),
                                     dst.location().deviceId());
    if (paths.isEmpty()) {
        // If there are no paths, flood and bail.
        flood(context, macMetrics);
        return;
    }
    ...
}
```

Then, the function installRule() tries to install the rule.

```
// Install a rule forwarding the packet to the specified port.
private void installRule(PacketContext context, PortNumber portNumber,
 ReactiveForwardMetrics macMetrics) {
 ...
    // If PacketOutOnly or ARP packet than forward directly to output port
    if (packetOutOnly || inPkt.getEtherType() == Ethernet.TYPE_ARP) {
        packetOut(context, portNumber, macMetrics);
        return;
    }
    ...
    flowObjectiveService.forward(context.inPacket().receivedFrom().deviceId(),
                                 forwardingObjective);
        forwardPacket(macMetrics);
}
```

4. [Control Plane] Controller sends a FLOW_MOD to s1 and installs the flow rule.
5. [Control Plane] Controller sends the packet(original) back to s1 with a PACKET-OUT message.
6. [Data Plane] s1 receives FLOW-MOD from the controller and installs the new flow rule in its flow table.
7. [Data Plane] s1 forwards the packet(original) to h2.
8. [Data Plane] After h2 receives the packet, it sends a reply(packet) to h1.
9. [Data Plane] h2 sends the packet to s1. Since s1 also doesn't have a flow rule for forwarding this packet, it generates a PACKET-IN message and sends the packet to the ONOS controller.
10. [Control Plane] Reactive Forwarding Application(org.onosproject.fwd) receives PACKET-IN and executes the function process() to decide the path of forwarding the packet.
11. [Control Plane] Controller sends a FLOW_MOD to s1 and installs the flow rule.
12. [Control Plane] Controller sends the packet(reply) back to s1 with a PACKET-OUT message.
13. [Data Plane] s1 receives FLOW-MOD from the controller and installs the new flow rule in its flow table.
14. [Data Plane] s1 forwards the packet(reply) to h1.

Part5

一開始在使用 curl 時沒辦法成功並會跳出下面的錯誤訊息

```
bash: ' http://localhost:8181/onos/v1/flows/of:0000000000000001 ' no such file or directory
```

後來嘗試過很多方法例如安裝 Java、修改 json 檔等，後來發現似乎是在 windows 將 url 複製貼上到 unbuntu terminal 時，格式會跑掉，於是重新手打 url 之後問題就解決了。

另外，我也在這次實驗中了解何謂 Broadcast Storm 以及該如何避免這個問題，同時也對怎麼使用 Wireshark 有了更深的了解。

References

Day 15 連結層攻擊實作- STP Spoofing - | iT 邦幫忙

https://ithelp.ithome.com.tw/articles/10247179

Flow Rules | ONOS Wiki

https://wiki.onosproject.org/display/ONOS/Flow+Rules

OpenFlow | Muzixing

https://www.muzixing.com/pages/2013/12/12/yuan-chuang-openflowtong-xin-liu-cheng-jie-du.html

apps/fwd/src/main/java/org/onosproject/fwd/ReactiveForwarding.java | GitHub

https://github.com/opennetworkinglab/onos/blob/onos-2.2/apps/fwd/src/main/java/org/onosproject/fwd/ReactiveForwarding.java